

~~return 0;~~

Date - 22nd June 2026

1) Linear Search

Linear search is a searching technique in which each element is checked one by one until the target element is found.

• Classification

1) Searching Algorithm

A searching algorithm is a method used to find a specific element within a collection of data.

2) Sequential Search

Sequential Search (another name for linear search) checks elements one by one from the beginning until the target element is found or the list ends.

3) works on Sorted and Unsorted Arrays.

Linear Search does not require the data to be arranged in any order. It can search successfully in both sorted and unsorted arrays.

• Algorithm

1. Start from first element.
2. Compare each element with target.
3. If match found $\rightarrow$ return index.
4. If end reached $\rightarrow$ Not Found.

• Time Complexity

Case	Complexity
Best	$O(1)$
Average	$O(n)$
Worst	$O(n)$

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int n, x;
```

```
    printf("Enter no. of students:");  
    scanf("%d", &n);
```

```
    int arr[n];
```

```
    printf("Enter arr numbers: \n");
```

```
    for(int i = 0; i < n; i++)
```

```
    {  
        scanf("%d", &arr[i]);
```

```
    }  
    printf("Enter arr number to search:");  
    scanf("%d", &x);
```

```
    int found = 0;
```

```
    for(int i = 0; i < n; i++)
```

```
    {  
        if(arr[i] == x)
```

```
        {  
            printf("Found at index %d", i);  
            found = 1;  
            break;
```

```
        }  
    }
```

```
if (found == 0)
```

```
{  
    printf("Not Found");
```

```
    return 0;  
}
```

main.c

Output



```
Enter number of students: 5
```

```
Enter roll numbers:
```

```
2
```

2. Binary Search

Binary Search finds an element by repeatedly dividing a sorted array into two halves.

• Classification

1) Searching Algorithm

A searching algorithm used to locate a target element in a dataset.

2) Divide and Conquer

Divide and Conquer is a technique that solves a problem by dividing it into smaller parts, solving the smaller parts, and combining the results. Binary search repeatedly divides the search space into two halves.

3) Requires Sorted Data

Binary search works correctly only when the elements are arranged in sorted order (ascending or descending).

Algorithm

1. Find middle element.
2. If target equals middle $\rightarrow$ Found.
3. If target $<$ middle $\rightarrow$ Search left half.
4. If target $>$ middle $\rightarrow$ Search right half.
5. Repeat until found or range becomes empty.

Time Complexity

Case Complexity

Best	$O(1)$
Average	$O(\log n)$
Worst	$O(\log n)$

```
#include <stdio.h>
```

SUNDAY 28

```
int main()
```

```
{
```

```
    int n, target;
```

```
    printf("Enter number of periods\n");  
    scanf("%d", &n);
```

2024

```
int product[n];
```

```
printf("Enter sorted product IDs\n");  
for (int i = 0; i < n; i++)
```

```
{  
    scanf("%d", &product[i]);  
}
```

```
printf("Enter product ID to search:\n");  
scanf("%d", &target);
```

```
int low = 0;
```

```
int high = n - 1;
```

```
int found = 0;
```

```
while (low <= high)
```

```
{  
    int mid = (low + high) / 2;  
    if (product[mid] == target)
```

```
{  
        found = 1;  
        break;  
    }
```

```
else if (product[mid] < target)
```

```
{  
        low = mid + 1;  
    }
```

```
else
```

```
{  
        high = mid - 1;  
    }
```

main.c

Output



Enter number of products: 10

Enter sorted product IDs:

5

5

3) Bubble Sort

Bubble Sort repeatedly compares adjacent elements and swaps them if they are in the wrong order.

• Classification

1) Sorting Algorithm

A sorting algorithm arranges data in a specific order, such as ascending or descending.

2) Comparison-Based Sort

A comparison-based sort determines the order of elements by comparing two elements and deciding which should come first.

3) Stable Sort

A stable sort preserves the relative order of elements having equal values after sorting.

Time Complexity

Case	Complexity
------	------------

Best	$O(n)$
------	--------

Average	$O(n^2)$
---------	----------

Worst	$O(n^2)$
-------	----------

Bubble Sort

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int n;
```

```
    printf("Enter number of scores:");  
    scanf("%d", &n);
```

```
    int score[n];
```

```
    printf("Enter scores: \n");  
    for (int i = 0; i < n; i++)
```

```
    {
```

```
        scanf("%d", &score[i]);  
    }
```

```
    for (int i = 0; i < n - 1; i++)
```

```
    {
```

```
        for (int j = 0; j < n - i - 1; j++)
```

```
        {
```

```
            if (score[j] > score[j+1])
```

```
            {
```

```
                int temp = score[j];
```

```
                score[j] = score[j+1];
```

```
                score[j+1] = temp;
```

```
            }
```

```
        }
```

```
    }
```

```
printf("Sorted Scores:\n");  
for (int i = 0; i < n; i++)  
{  
    printf("%d", score[i]);  
}
```

```
} return 0;
```

main.c

Output



Enter number of scores: 50

Enter scores:

40

4) Insertion Sort
Insertion sort inserts each element into its correct position in the already sorted part of the array.

• Classification.

1) Sorting Algorithm
A method that arranges data in a desired order.

2) Comparison-Based Sort
It sorts elements by comparing them with previously sorted elements and placing them in the correct position.

3) Stable Sort
Insertion Sort maintains the original order of equal elements during sorting.

Time Complexity	
Case	Complexity
Best	$O(n)$
Average	$O(n^2)$
Worst	$O(n^2)$

```

int main ()
{
    int n;
    printf ("Enter number of books: ");
    scanf ("%d", &n);

    int books [n];
    printf ("Enter book numbers: \n");
    for (int i = 0; i < n; i++)
    {
        scanf ("%d", &books [i]);
    }

    for (int i = 1; i < n; i++)
    {
        int key = books [i];
        int j = i - 1;

        while (j >= 0 && books [j] > key)
        {
            books [j+1] = books [j];
            j--;
        }
        books [j+1] = key;
    }

    printf ("Sorted Book Numbers: \n");
    for (int i = 0; i < n; i++)
    {
        printf ("%d", books [i]);
    }
}

```

main.c

Output



```
Enter number of books: 50
```

```
Enter book numbers:
```

```
5
```

5) Selection Sort

Selection Sort repeatedly selects the smallest element and places it at the beginning.

• Classification.

1) Sorting Algorithms

An algorithm that arranges elements in ascending or descending order.

2) Comparison-Based Sort

It repeatedly compares elements to find the minimum (or maximum) value and places it in its correct position.

3) In-place Sort

04 SUNDAY An in-place sort requires only a small, constant amount of extra memory and performs sorting within the original array itself.

~~4) A.A.~~

Complexity

Case	Complexity
Best	$O(n^2)$
Average	$O(n^2)$
Worst	$O(n^2)$

```
#include <stdio.h>

int main ()
{
    int n;

    printf("Enter number of players:");
    scanf("%d", &n);

    int score[n];

    printf("Enter player scores: \n");
    for (int i = 0; i < n; i++)
    {
        scanf("%d", &score[i]);
    }

    for (int i = 0; i < n - 1; i++)
    {
        int minIndex = i;

        for (int j = i + 1; j < n; j++)
```

```
1 { if (score[i] < score[minIndex])  
2 {  
3     minIndex = i;  
4 }  
5 }
```

```
6 int temp = score[i];  
7 score[i] = score[minIndex];  
8 score[minIndex] = temp;  
9 }
```

```
10 printf("Sorted Scores: \n");  
11 for (int i = 0; i < n; i++)  
12 {  
13     printf("%d", score[i]);  
14 }
```

```
15 return 0;  
16 }
```

main.c

Output



Enter number of players: 11

Enter player scores:

5